# Reth 심층 분석: 모듈형 Rust Ethereum 실행 클라이언트

## 1. 개요

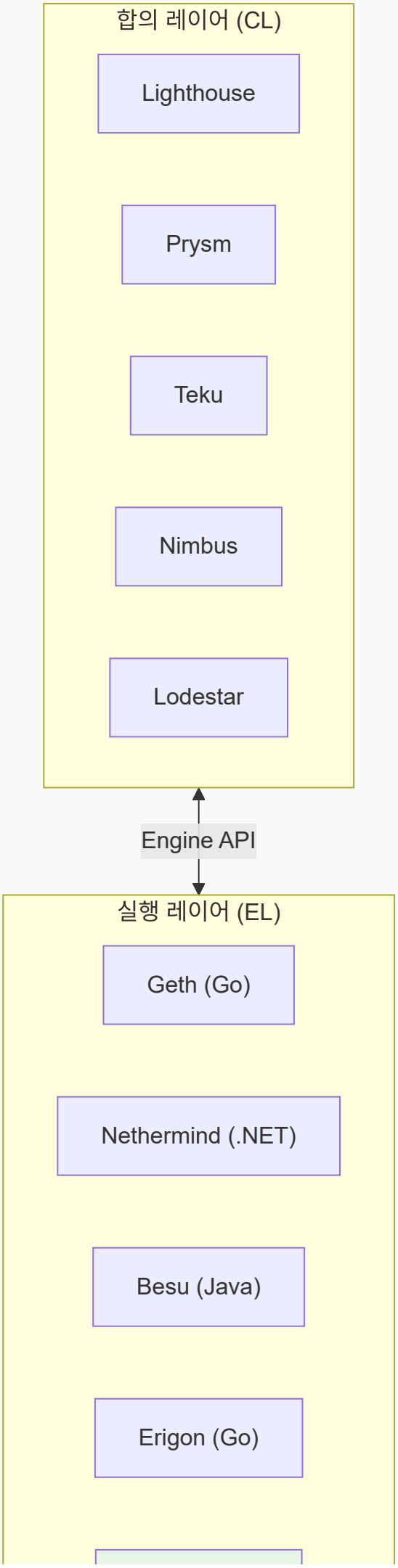
### 1.1 Reth란?

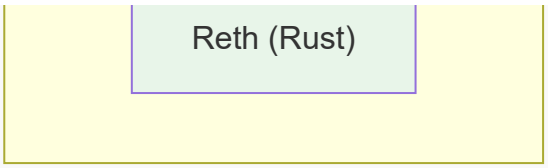
Reth(Rust Ethereum)는 Paradigm이 개발한 **Rust 기반 Ethereum 실행 클라이언트**입니다. "실행 클라이언트"란 Ethereum의 트랜잭션 실행, 상태 관리, JSON-RPC 제공 등을 담당하는 노드 소프트웨어를 말합니다.

> “Reth is a new Ethereum full node implementation that is focused on being user-friendly, highly modular, as well as being fast and efficient.”
> — Reth README

### 1.2 Ethereum 클라이언트 구조

Ethereum은 **The Merge** 이후 실행 레이어(EL)와 합의 레이어(CL)가 분리되었습니다.


합의 레이어 (CL)
Lighthouse
Prysm
Teku
Nimbus
Lodestar
Engine API
실행 레이어 (EL)
Geth (Go)
Nethermind (.NET)
Besu (Java)
Erigon (Go)




### 1.3 클라이언트 다양성의 중요성

단일 클라이언트가 66% 이상의 점유율을 가지면 치명적 버그 발생 시 네트워크 finality가 중단되거나 체인 분기가 발생할 수 있습니다.

> "The Ethereum protocol becomes more antifragile when no node implementation dominates. This ensures that if there's a software bug, the network does not finalize a bad block."
> — Reth README

### 1.4 Reth의 목표

1. **모듈성**: 모든 컴포넌트를 독립 라이브러리(crate)로 제공
2. **성능**: Rust + Staged Sync + MDBX로 최고 성능 달성
3. **클라이언트 다양성**: Ethereum 네트워크의 반취약성 기여
4. **다중 체인 지원**: Ethereum, Optimism, BSC, Polygon 등 지원
5. **구성 가능성**: 다양한 사용 사례에 맞춤

---

## 2. Geth vs Reth: 설계 철학 비교

### 2.1 언어 및 아키텍처

| 구분 | Geth | Reth |
|---|---|---|
| **언어** | Go | Rust |
| **시작 연도** | 2014 | 2022 |
| **아키텍처** | 통합형 | 모듈형 (Crate 기반) |
| **동기화 방식** | Snap Sync | Staged Sync (Erigon 방식) |
| **데이터베이스** | LevelDB | MDBX |
| **상태 저장** | Merkle Patricia Trie | Flat Tables + MPT |
| **라이선스** | LGPL-3.0 | Apache 2.0 + MIT |
| **라이브러리 사용** | 어려움 | 설계 목표 |

### 2.2 성능 비교

**벤치마크 결과**:

- 동기화 속도: Geth 대비 ~4x 빠름
- 아카이브 노드 저장소: Geth ~12+ TB vs Reth ~2 TB
- BSC-Reth: Geth 대비 300% 성능 향상
- 목표 처리량: 1G gas/sec @ 750ms 블록 타임

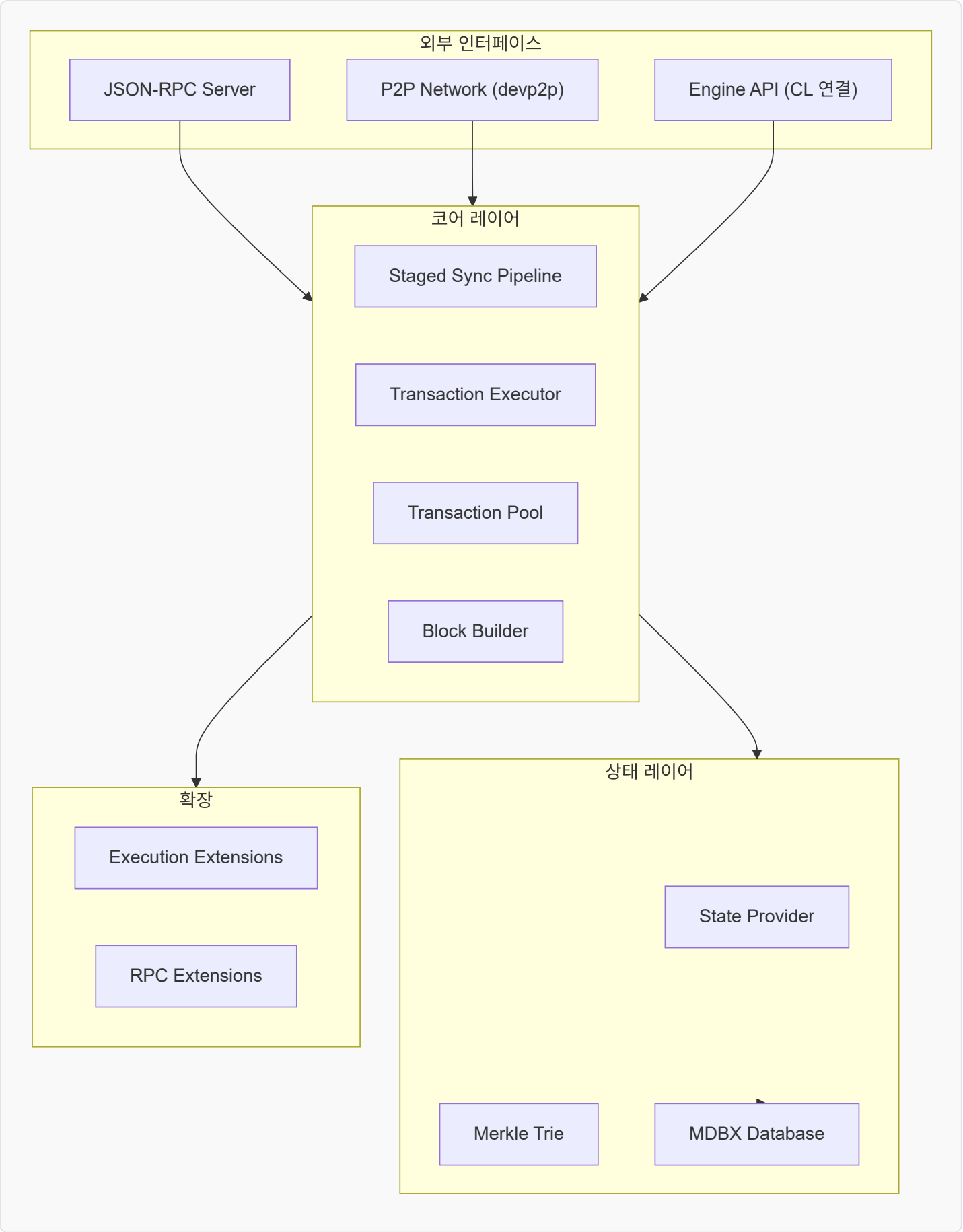
### 2.3 설계 철학 차이

| Geth | Reth |
|---|---|
| 10년+ 검증된 안정성 | 모듈성 우선 |
| 참조 구현 | 라이브러리로 사용 가능 |
| 보수적 변경 | 최신 연구 적용 |
| 광범위한 채택 | 빠른 혁신 |

---

## 3. Reth 아키텍처

### 3.1 전체 구조




## 3.2 Crate 구조

```
reth/
├── bin/reth/                    # CLI 바이너리
├── crates/
│   ├── reth/                     # 메인 노드 진입점
```

```
│    │
│    ├── 데이터베이스
│    │    ├── reth-db/            # MDBX 구현
│    │    ├── reth-db-api/        # DB trait 정의
│    │    └── reth-db-models/     # 데이터 모델
│    │
│    ├── 동기화
│    │    ├── reth-stages/        # Staged Sync 파이프라인
│    │    ├── reth-stages-api/    # Stage trait 정의
│    │    └── reth-pipeline/      # 파이프라인 오케스트레이션
│    │
│    ├── 실행
│    │    ├── reth-evm/           # EVM 추상화
│    │    ├── reth-revm/          # revm 통합
│    │    └── reth-execution-types/# 실행 타입
│    │
│    ├── 네트워킹
│    │    ├── reth-network/       # P2P 네트워크
│    │    ├── reth-eth-wire/      # Ethereum Wire Protocol
│    │    └── reth-discv4/        # Node Discovery v4
│    │
│    ├── RPC
│    │    ├── reth-rpc/           # JSON-RPC 서버
│    │    ├── reth-rpc-api/       # RPC trait 정의
│    │    └── reth-rpc-types/     # RPC 타입
│    │
│    ├── 상태
│    │    ├── reth-provider/      # 상태 제공자
│    │    ├── reth-trie/          # Merkle Trie
│    │    └── reth-state/         # 상태 관리
│    │
│    ├── 합의
│    │    ├── reth-consensus/     # 합의 검증
│    │    └── reth-engine/        # Engine API
│    │
│    ├── 확장
│    │    ├── reth-exex/          # Execution Extensions
│    │    └── reth-node-builder/  # 노드 빌더
│    │
│    └── 프리미티브
│        ├── reth-primitives/    # 기본 타입
│        └── reth-codecs/        # 인코딩/디코딩
│
└── examples/                   # 예제 코드
```

### 3.3 핵심 의존성

- **revm**: Paradigm의 Rust EVM 구현, Foundry에서 검증됨
- **alloy**: 차세대 Rust Ethereum 라이브러리 (ethers-rs 후속)
- **libmdbx**: Erigon이 도입한 고성능 key-value 데이터베이스

- **tokio**: 비동기 런타임

---

# 4. Staged Sync 아키텍처

## 4.1 Staged Sync란?

Erigon(구 Turbo-Geth)이 개척한 **단계별 동기화 방식**입니다. 전체 동기화를 독립적인 단계(stage)로 분리하여 각 단계를 개별적으로 실행, 중단, 재개할 수 있습니다.

## 4.2 기존 vs Staged Sync 비교

| 구분 | 기존 (Geth Snap Sync) | Staged Sync (Reth) |
|---|---|---|
| **구조** | 모놀리식 | 단계별 분리 |
| **중단 복구** | 처음부터 재시작 | 해당 단계부터 재개 |
| **병렬화** | 제한적 | 단계별 최적화 가능 |
| **디버깅** | 어려움 | 단계별 검사 가능 |
| **진행 추적** | 제한적 | 단계별 체크포인트 |

## 4.3 각 Stage 상세

### 4.3.1 HeadersStage

- P2P에서 블록 헤더 다운로드
- PoW/PoS 규칙에 따른 헤더 검증
- DB에 헤더 저장

### 4.3.2 BodiesStage

- 헤더에서 body hash 추출
- P2P에서 블록 본문 다운로드
- 트랜잭션 루트, 언클 검증

### 4.3.3 SendersStage

- 트랜잭션에서 서명자 주소 복구 (ECDSA)

- Execution 단계에서 재계산 방지

#### 4.3.4 ExecutionStage (가장 무거운 단계)

- 모든 트랜잭션을 EVM에서 실행
- 상태 변경(changeset) 생성
- 설정된 임계값마다 중간 커밋

```
[stages.execution]
max_blocks = 500000
max_changes = 5000000
max_cumulative_gas = 1500000000
max_duration = { secs = 10, nanos = 0 }
```

#### 4.3.5 MerkleStage

- 상태 루트 계산
- 증분 업데이트 또는 전체 재구축 선택

#### 4.3.6 Hashing Stages

- AccountHashingStage: 계정 해시 계산
- StorageHashingStage: 스토리지 해시 계산

#### 4.3.7 History Stages

- IndexAccountHistoryStage: 계정 히스토리 인덱싱
- IndexStorageHistoryStage: 스토리지 히스토리 인덱싱
- TransactionLookupStage: TX 해시 → TX 번호 인덱스

### 4.4 Unwind (Reorg 처리)

체인 재구성 시 각 단계를 **역순으로** unwind합니다:

```
Forward: Headers → Bodies → Senders → Execution → Merkle → ...
Unwind:  ... → Merkle → Execution → Senders → Bodies → Headers
```

---

## 5. 데이터베이스 설계

### 5.1 MDBX 선택 이유

| 특성 | LevelDB (Geth) | MDBX (Reth) |
|---|---|---|
| **구조** | LSM Tree | B+ Tree |
| **읽기 성능** | 중간 | 높음 |
| **메모리 매핑** | 미지원 | 지원 |
| **트랜잭션** | 제한적 | 완전한 ACID |
| **Copy-on-Write** | 미지원 | 지원 |
| **동시 읽기** | 제한적 | 무제한 |

## 5.2 테이블 스키마

**블록 관련 테이블**:

- `CanonicalHeaders`: BlockNumber → HeaderHash
- `Headers`: BlockNumber → Header
- `BlockBodyIndices`: BlockNumber → (first_tx_num, tx_count)
- `Transactions`: TxNumber → TransactionSigned
- `Receipts`: TxNumber → Receipt

**상태 관련 테이블**:

- `PlainAccountState`: Address → Account (빠른 조회용)
- `PlainStorageState`: (Address, Key) → Value (빠른 조회용)
- `HashedAccounts`: keccak256(Address) → Account (Merkle 계산용)
- `HashedStorages`: (keccak256(Address), keccak256(Key)) → Value

**Trie 관련 테이블**:

- `AccountsTrie`: Path → BranchNode
- `StoragesTrie`: (HashedAddress, Path) → BranchNode

**히스토리 테이블**:

- `AccountChangeSets`: BlockNumber → 변경된 계정들
- `StorageChangeSets`: BlockNumber → 변경된 스토리지들

## 5.3 Flat Storage vs Trie Storage

Reth는 **Flat Storage + Trie 분리** 전략을 사용합니다:

- **Plain***: 빠른 상태 조회 (랜덤 액세스)
- **Hashed***: Merkle 계산용 (정렬된 액세스)

- **Trie**: 상태 루트 계산용

이 분리로 각 용도에 최적화된 인덱싱이 가능합니다.

### 5.4 Compact Encoding

저장 공간 최적화를 위한 커스텀 인코딩:

```
#[derive(Compact)]
struct Account {
    nonce: u64,
    balance: U256,
    bytecode_hash: Option<B256>,
}
```

- 불필요한 0 바이트 제거
- 옵션 필드 비트플래그로 표시
- 가변 길이 정수 인코딩

---

## 6. Execution Extensions (ExEx)

### 6.1 ExEx 개요

**Execution Extensions (ExEx)**는 Reth의 혁신적인 확장 메커니즘입니다. 블록 실행 후 상태 변경을 구독하여 커스텀 로직을 실행할 수 있습니다.

> "Execution Extensions (ExExes) are post-execution hooks for building real-time, high performance and zero-operations off-chain infrastructure on top of Reth."
> — Paradigm

### 6.2 ExEx vs 기존 방식

| 구분 | 기존 방식 | ExEx |
|---|---|---|
| **아키텍처** | 별도 프로세스 + JSON-RPC | 같은 프로세스, 공유 메모리 |
| **지연** | 네트워크 I/O | 제로 |
| **Reorg 처리** | 수동 구현 | 내장 (ChainReorged 이벤트) |
| **운영** | 다중 서비스 관리 | 단일 바이너리 |
| **코드량** | 많음 | 10x 이상 감소 |

### 6.3 ExExNotification 구조

```
pub enum ExExNotification {
    /// 체인에 커밋된 블록들
    ChainCommitted { new: Arc<Chain> },

    /// 체인 재구성 (Reorg)
    ChainReorged { old: Arc<Chain>, new: Arc<Chain> },

    /// 체인 되돌림
    ChainReverted { old: Arc<Chain> },
}

pub struct Chain {
    blocks: Vec<SealedBlockWithSenders>,
    execution_outcome: ExecutionOutcome,
    trie_updates: Option<TrieUpdates>,
}
```

## 6.4 ExEx 구현 예시

```
async fn transaction_counter<Node: FullNodeComponents>(
    mut ctx: ExExContext<Node>,
) -> eyre::Result<()> {
    while let Some(notification) = ctx.notifications.try_next().await? {
        match &notification {
            ExExNotification::ChainCommitted { new } => {
                for block in new.blocks() {
                    println!("Block {}: {} txs",
                        block.number,
                        block.body.transactions.len());
                }
            }
            ExExNotification::ChainReorged { old, new } => {
                println!("Reorg: {} removed, {} added",
                    old.blocks().count(),
                    new.blocks().count());
            }
            _ => {}
        }

        // Pruning을 위해 처리 완료 알림
        if let Some(committed) = notification.committed_chain() {
            ctx.events.send(ExExEvent::FinishedHeight(
                committed.tip().number
            ))?;
        }
    }
    Ok(())
}
```

### 6.5 ExEx 사용 사례

| 사용 사례 | 설명 | 코드량 |
|---|---|---|
| **Reorg Tracker** | 체인 재구성 추적 | ~20 LoC |
| **Indexer** | 블록/TX/이벤트 인덱싱 | ~250 LoC |
| **Rollup** | L2 상태 파생 | ~1000 LoC |
| **MEV Bot** | MEV 기회 탐지 | 가변 |
| **Bridge Monitor** | 크로스체인 모니터링 | 가변 |
| **Shadow Fork** | 섀도우 시뮬레이션 | 가변 |

---

## 7. Reth SDK: 커스텀 체인 구축

### 7.1 Reth SDK 개요

Reth는 단순한 노드 클라이언트가 아닌 **SDK**로도 사용할 수 있습니다.

### 7.2 커스터마이징 포인트

1. **커스텀 트랜잭션 타입**: 새로운 EIP-2718 타입 추가
2. **합의 규칙**: 블록/헤더 검증 커스터마이징
3. **Mempool 정책**: 트랜잭션 선택 로직
4. **RPC 확장**: 커스텀 메서드 추가
5. **프리컴파일**: 새로운 프리컴파일 추가
6. **Gas 모델**: 수수료 메커니즘 변경

### 7.3 채택 사례

| 프로젝트 | 사용 방식 | 특징 |
|---|---|---|
| **Berachain** | Bera-Reth | 가스리스 시스템 TX, 최소 base fee |
| **Optimism** | OP-Reth | L2 특화 기능 |
| **BSC** | BSC-Reth | 300% 성능 향상 |
| **Tempo** | Reth SDK | 결제 특화 기능 |
| **Circle Arc** | Reth SDK | 스테이블코인 체인 |

### 7.4 SDK vs Fork 비교

| 구분 | Geth Fork | Reth SDK |
|---|---|---|
| **업스트림 동기화** | 수동, 어려움 | 자동 |
| **코드 변경량** | 전체 | 필요한 부분만 |
| **유지보수** | 높음 | 낮음 |
| **커뮤니티 지원** | 제한적 | Reth 커뮤니티 활용 |

---

## 8. 운영 가이드

### 8.1 설치 및 실행

```
# 바이너리 설치
curl -L https://github.com/paradigmxyz/reth/releases/latest/download/reth-x86_64-unknown-linux-gnu.ta

# 노드 실행
reth node ₩
    --chain mainnet ₩
    --datadir /data/reth ₩
    --http ₩
    --http.api eth,web3,net,debug,trace
```

### 8.2 권장 하드웨어

| 모드 | CPU | RAM | 저장소 |
|---|---|---|---|
| **풀 노드** | 4+ 코어 | 8+ GB | 1+ TB NVMe |
| **아카이브 노드** | 8+ 코어 | 16+ GB | 2+ TB NVMe |
| **MEV/빌더** | 16+ 코어 | 32+ GB | 2+ TB NVMe |

### 8.3 알려진 제한사항

1. `eth_getRawTransactionByHash`: 미지원
2. `trace_filter`: 개발 중
3. Lighthouse와 일부 호환성 이슈 (Nimbus 권장)

---

## 9. 결론

### 9.1 Reth의 강점

- **모듈형 아키텍처**: 라이브러리로 사용 가능
- **고성능**: Staged Sync + MDBX
- **메모리 안전성**: Rust
- **ExEx 확장성**: 포스트-실행 훅
- **클라이언트 다양성**: Ethereum 네트워크 기여

### 9.2 사용 권장

| 사용 사례 | 권장 | 이유 |
|---|---|---|
| RPC 운영 | Reth | 더 빠름 |
| 아카이브 노드 | Reth | 저장소 효율 |
| 인덱서 구축 | Reth | ExEx 활용 |
| 커스텀 체인 | Reth | SDK 제공 |

---

## 참고 자료

### 공식 자료

- **문서**: https://reth.rs
- **GitHub**: https://github.com/paradigmxyz/reth
- **API 문서**: https://reth.rs/docs/

### 관련 프로젝트

- **revm**: https://github.com/bluealloy/revm
- **alloy**: https://github.com/alloy-rs/alloy
- **Foundry**: https://github.com/foundry-rs/foundry